



Universidad de Valladolid

Escuela Técnica Superior de Ingenieros de
Telecomunicación

Configuración y actualización dinámica de modelos
de aprendizaje profundo

GRADO EN INGENIERÍA DE TECNOLOGÍAS DE LA TELECOMUNICACIÓN

TRABAJO DE FIN DE GRADO

Autor:
Andrés García Treviño

Tutor:
Dr. D. Pablo Casaseca de la Higuera

Valladolid, junio 2026

1. Introducción

Las redes de telecomunicaciones tienen como finalidad el intercambio de información entre un nodo transmisor y un nodo receptor. Es un concepto sencillo afectado por un problema de números. Siguiendo la ley de Bell, el número de dispositivos ha aumentado exponencialmente desde 1 ordenador básico en un departamento de una empresa a múltiples dispositivos por persona (teléfonos móviles, relojes inteligentes...) [1]. Casi con la misma tendencia ha evolucionado la tecnología de las telecomunicaciones, impidiendo que el crecimiento de los dispositivos saturase las capacidades de la red. De esta forma, en la actualidad existe una infinidad de nodos transmisores y receptores.

Los flujos de información intercambiada entre los nodos transmisores y receptores forma el tráfico de red. Dicho tráfico puede ser representados y medidos. Las medidas o muestras realizadas en diferentes instantes de tiempo pueden aunarse en una serie temporal. Como todas las series, cabe la posibilidad de que sigan un patrón que pueda utilizarse para predecir medidas futuras a partir del histórico de medidas pasadas [2]. Con la inmensa cantidad de flujos de información existentes a día de hoy, resulta imposible analizarlos mediante la intervención humana. Sin embargo, la situación planteada representa el caso de uso de la inteligencia artificial y *Deep Learning*: averiguar el patrón subyacente de un conjunto de datos [3].

La predicción del tráfico de red tiene numerosísimas aplicaciones como, por ejemplo, la correcta planificación y organización de los recursos de las redes de telecomunicaciones. Esto se traduce en un menor desperdicio de recursos, que a su vez se traduce en una inversión económica más eficiente, sin menoscabar la calidad del servicio proporcionado por la red de telecomunicaciones [4]. También es posible aplicar la predicción de tráfico de red a tareas como la detección de anomalías o intrusos, monitorización o mantenimiento [5].

Sin embargo, en la actualidad existen multitud de aplicaciones que generan flujos de datos a demanda. Algunos ejemplos son el *media streaming* o los modelos de negocio *as a Service* (XaaS) [6]. Los modelos de predicción estáticos convencionales no son capaces de adaptarse al dinamismo y la falta de estacionariedad de dichos flujos de datos.

Para solucionar dicho problema se han desarrollado modelos capaces de adaptarse a los cambios en las tendencias o patrones subyacentes en los flujos de tráfico. Estos modelos se basan en *online active deep learning*. De esta forma se permite reentrenar los modelos a medida que se incorporan nuevas muestras para adaptarse a ellas en un proceso llamado *concept drift* [7].

Este trabajo se basa en la investigación realizada en [8] acerca de modelos LSTM que implementan actualizaciones incrementales donde las muestras sobre las que los modelos tuvieron un mayor error toman mayor importancia. En este trabajo se plantea la hipótesis de que un modelo que combine arquitecturas de redes neuronales convolucionales y de redes neuronales recurrentes tenga una precisión predictiva en *online learning* superior a aquellos modelos basados únicamente en redes neuronales recurrentes.

1.1. Objetivos

El principal objetivo de este trabajo es analizar la capacidad de adaptación mediante entrenamiento incremental de diferentes modelos de redes neuronales convolucionales recurrentes ante cambios en el patrón subyacente de series temporales.

Dentro del objetivo principal se establecen los siguientes objetivos secundarios:

- La comprensión y dominio del marco teórico utilizado en este trabajo.
- El manejo de herramientas de desarrollo como Python o Kaggle.
- La mejora del modelo base proporcionado.
- Evaluar el impacto producido por las redes neuronales convolucionales respecto al modelo base.

1.2. Estructura del documento

Esta memoria se divide en 5 capítulos: introducción, antecedentes y estado del arte, entorno experimental, exposición de resultado y discusión y conclusiones y líneas futuras.

El Capítulo 1 se desarrolla la motivación y los objetivos de este trabajo. Además, se plantean la hipótesis y los objetivos y se añade una breve descripción de la estructura de la memoria.

El Capítulo 2 ofrece una visión general del marco teórico relevante a la predicción de series temporales, el aprendizaje automático, *deep learning*, *online learning* y las diferentes arquitecturas de redes neuronales convolucionales y recurrentes.

El Capítulo 3 presenta la base de datos utilizada, las arquitecturas analizadas y los criterios de selección y evaluación de modelos de aprendizaje automático aplicados en este trabajo.

El Capítulo ?? muestra y analiza el rendimiento de los distintos modelos propuestos. También interpreta los resultados obtenidos y comenta las relaciones presentes entre modelos y resultados.

El Capítulo ?? resume las aportaciones de este trabajo y presenta las conclusiones obtenidas. También propone líneas de actuación futuras.

2. Antecedentes y estado del arte

2.1. Predicción de series temporales

Una serie temporal puede ser representada como un vector unidimensional en cuyas posiciones se encuentran muestras secuenciales y equiespaciadas en el tiempo de una misma variable. Existen multitud de métodos de predicción de series de datos basados en la extrapolación. Sin embargo, no son específicos de series temporales sino que se aplican a conjuntos de datos genéricos.

El estudio clásico de las series temporales considera que su comportamiento es fruto de la tendencia, las variaciones cíclicas y estacionales y un componente aleatorio. La tendencia es la componente que refleja la evolución lineal a largo plazo de la serie temporal de forma lineal, exponencial... La componente cíclica recoge las oscilaciones periódicas de amplitud superior a un año. Por último, la componente estacional recoge las oscilaciones periódicas de amplitud inferiores a un año como las oscilaciones mensuales, semanales, diarias... Estas tendencias pueden generar una combinación aditiva si son independientes entre sí o multiplicativa si están correlacionadas entre sí [9].

El estudio de las series temporales ha evolucionado con la aparición del aprendizaje automático. Los modelos clásicos autorregresivos basados en medias móviles siguen un algoritmo concreto. Los modelos de aprendizaje automático se adaptan al patrón subyacente de las series temporales por sí mismos.

2.1.1. Modelos autorregresivos de medias móviles

Como su propio nombre indica, los modelos autorregresivos de medias móviles para la predicción de series temporales están basados en los modelos de medias móviles y autorregresivos. Las medias móviles son una manera simple de obtener una predicción cuando no hay una tendencia marcada ni tampoco estacionalidad. Se utiliza el término medias móviles puesto que conforme se va disponiendo de nuevas observaciones se pueden calcular nuevas medias mediante la inclusión de las nuevas muestras y el desprendimiento de las más antiguas [10]. De esta forma se eliminan las oscilaciones de mayor frecuencia, actuando como un filtro paso bajo.

Las medias móviles unilaterales permiten estimar el valor futuro de la serie temporal $\hat{Y}_t$ como la media de las m muestras inmediatamente anteriores Y_t :

$$\hat{Y}_t = \frac{1}{m} * \sum_{i=1}^m Y_{t-i} \quad (1)$$

Asimismo, la media móvil puede combinarse con una ponderación k_i . La ponderación más habitual es la exponencial asumiendo que las muestras más recientes contribuyen más a la correcta predicción que las muestras más antiguas, modificando la ecuación

1 de la forma:

$$\hat{Y}_t = \frac{1}{m} * \sum_{i=1}^m Y_{t-i} * k_{t-i}$$

El modelo ARMA (*AutoRegressive Moving Average*) es el modelo más sencillo que combina las medias móviles t la autorregresión. Utiliza tanto las muestras reales de la serie temporal Y_t como los errores cometidos en predicciones pasadas ϵ_t . Concretamente, la cantidad de muestras y errores cometidos está estipulada por los parámetros p y q respectivamente. Siendo α_t y θ_t la ponderación aplicada a cada muestra y error [11–13]:

$$\begin{aligned}\hat{Y}_t &= \sum_{i=1}^p \alpha_i * Y_{t-i} + \sum_{i=1}^q \theta_i * \epsilon_{t-i} \\ \epsilon_t &= Y_t - \hat{Y}_t\end{aligned}$$

El primer sumatorio es la parte autorregresiva, realiza una regresión basada en datos pasados. El segundo sumatorio es la media móvil de los errores cometidos en predicciones pasadas.

ARMA fue desarrollada inicialmente para la predicción de datos financieros [14] pero ha dado grandes resultados en sus adaptaciones a la predicción de tráfico en red a corto plazo [15, 16]. Sin embargo, el modelo ARMA presupone que la serie temporal es estacionaria, es decir, que sus propiedades estadísticas son constantes en el tiempo, sin tendencias ni comportamientos periódicos [11].

El modelo ARIMA (*AutoRegressive Integrated Moving Average*) introduce restas como instrumento para eliminar las tendencias presentes en la serie temporal. Al restar el valor de una muestra al valor de la siguiente se eliminan las tendencias lineales. Aplicar una segunda resta elimina las tendencias cuadráticas, etc. De esta forma, la serie temporal resultante es estacionaria.

Tomando una serie temporal de ejemplo [1, 4, 9, 16, 25, 36] con una tendencia cuadrática, tras una etapa de resta se obtiene la serie [3, 5, 7, 9, 11] con una tendencia lineal. Tras una segunda ronda de resta aplicada sobre la serie lineal se obtiene la serie estacionaria [2, 2, 2, 2]. La cantidad de rondas de restas viene especificada en el parámetro d [11, 16].

$$\begin{aligned}d = 0 & \rightarrow [1, 4, 9, 16, 25, 36] \\ d = 1 & \rightarrow [4 - 1, 9 - 4, 16 - 9, 25 - 16, 36 - 25] = [3, 5, 7, 9, 11] \\ d = 2 & \rightarrow [5 - 3, 7 - 5, 9 - 7, 11 - 9] = [2, 2, 2, 2]\end{aligned}$$

El modelo ARIMA sólo tiene en cuenta tendencias pero no comportamientos cíclicos o estacionales. El modelo SARIMA (*seasonal ARIMA*) utiliza el mismo mecanismo que ARIMA pero modelando la componente estacional para una determinada frecuencia $1/s$. Además de realizar la resta con respecto a la muestra anterior, resta también la muestra en $t - s$. De esta forma, SARIMA utiliza los parámetros p , d y q para modelar la tendencia y los parámetros P , D , Q y s para modelar la componente estacional/cíclica [11, 17].

2.2. Aprendizaje automático

Existen varias definiciones para el aprendizaje automático. Un programa aprende de la experiencia E con respecto a una tarea T y una métrica de rendimiento R si su rendimiento en la tarea, medido por R , mejora con más experiencia E [18]. El aprendizaje automático tiene como objetivo crear sistemas capaces de aprender por ellos mismos a partir de un conjunto de datos sin ser programados de forma explícita [19].

Para resolver un problema de aprendizaje automático es necesario que haya datos con los que conformar la experiencia E , que exista un patrón en dichos datos y, lo más importante, que no exista un modelo matemático que resuelva el problema directamente. Mediante diferentes algoritmos de aprendizaje como la regresión, los árboles de decisión o *k-means clustering*, el programa puede aprender el patrón subyacente en los datos.

Dentro del aprendizaje automático se encuentran dos grandes grupos: el aprendizaje no supervisado y el aprendizaje supervisado. El aprendizaje no supervisado utiliza datos no etiquetados en los que no hay una relación entre los datos de entrada y la salida esperada para realizar tareas como *clustering* o detección de anomalías. Por el contrario, el aprendizaje supervisado utiliza la relación existente y conocida entre los datos de entrada y la salida esperada para resolver problemas de clasificación y regresión.

Las series temporales son datos etiquetados por su propia naturaleza. Una muestra utilizada como dato de entrada tiene a la siguiente muestra como salida esperada. La predicción de series temporales es en sí un problema de regresión. Por lo tanto, entra dentro del aprendizaje automático supervisado para problemas de regresión [20].

2.2.1. Procedimiento de selección de modelos

El aprendizaje automático funciona encontrando el conjunto de valores de las variables utilizadas por el modelo que producen los resultados más precisos. No siempre se consideran todas las variables en la búsqueda del mejor conjunto de valores. Las variables que sí son consideradas se denominan hiperparámetros.

Existen diferentes estrategias en la búsqueda de los mejores hiperparámetros. Entre las estrategias más comunes se encuentran el *grid search* y el *random search*. La

primera itera sobre un conjunto de valores concretos establecidos mientras la segunda utiliza conjuntos de valores obtenidos de forma aleatoria dentro de un rango. *Grid search* ofrece un gran control sobre los valores utilizados en la búsqueda mientras *random search* ofrece la posibilidad de encontrar buenos conjuntos de valores en un rango de búsqueda más amplio. Ninguna estrategia es infalible, siempre cabe la posibilidad de que el modelo para el que se realiza la búsqueda no sea un buen modelo [21].

Según cómo se diseñe el programa o modelo de aprendizaje automático y el valor de sus hiperparámetros, su rendimiento a la hora de obtener el patrón subyacente a unos datos puede variar mucho. A mayores, no existe *a priori* un modelo mejor que otro [22]. Por ello, se ha diseñado un protocolo de creación de modelos de aprendizaje automático.

El aprendizaje automático se divide en tres fases que contienen 3 sets de datos diferentes: entrenamiento, validación y test. El entrenamiento refiere a la primera fase, donde el programa o modelo aprende en la medida que le es posible el patrón subyacente presente en los datos de entrenamiento. Como se verá más adelante, esta fase involucra algoritmos como *backpropagation* o *gradient descent*. El grado de aprendizaje correcto del patrón de los datos de entrenamiento se denomina error de entrenamiento o E_{in} , siendo 0 el mejor valor posible.

El modelo, una vez pasada la fase de entrenamiento, dejará de aprender para convertirse en un modelo en producción que se dedicará a aplicar el patrón aprendido en el entrenamiento a nuevas muestras. Sin embargo, aunque el modelo tenga un E_{in} bajo, puede no dar las respuestas correctas a las nuevas muestras. El error cometido en las muestras en producción se denomina como E_{out} y el error de generalización se calcula como $E_{out} - E_{in}$. Para evitar que esto ocurra y se lance un modelo a producción que no funcione bien, provocando en la mayoría de casos pérdidas económicas y oportunidades de negocio, se introducen las fases de validación y test.

No se pueden comparar modelos entre sí directamente usando E_{in} puesto que esto sólo conllevaría a la elección del modelo con menor E_{in} y, en consecuencia, con mayor E_{out} . Esta situación es conocida como *overfitting*. La fase de validación se encarga de comparar los diferentes modelos propuestos haciéndoles procesar un set de datos de validación como si estuvieran en producción. El desempeño de los modelos en esta fase o E_{val} sí que puede utilizarse para comparar modelos.

Sin embargo la fase de validación tiene el mismo problema de antes. Cabe la posibilidad de que el modelo escogido haya realizado *overfitting* en los datos de validación y, aunque tenga un valor pequeño de E_{val} , tenga un gran error E_{out} . Por ello, la fase de test reentrena el modelo utilizando los datos de entrenamiento y validación y mide el nuevo rendimiento con un set de datos de test. El error obtenido E_{test} es entonces una buena estimación de E_{out} .

Es importante recalcar que los datos de test sólo deben ser utilizados en la fase de test. Utilizarlos en otras fases como la de entrenamiento o en el reentrenamiento del modelo tras la validación provocará una distorsión en la medida E_{test} llamada *data leakage*. Tampoco ha de usarse para comparar modelos como si fuera un test de

validación [11, 20, 23].

2.3. *Deep learning*

Deep learning es considerado un subconjunto dentro del aprendizaje automático. Mientras el aprendizaje automático o *machine learning* utiliza algoritmos de aprendizaje diseñados por humanos, *deep learning* utiliza estructuras neuronales para extraer el patrón subyacente en los datos de forma directa sin necesidad de algoritmos concretos como árboles de decisión o *clustering*.

Con el objetivo de crear un programa que imitara la inteligencia humana nacieron las neuronas artificiales que imitan el funcionamiento de las neuronas biológicas del cerebro humano. Las neuronas artificiales imitan la sinapsis biológica de tal forma que evalúan la cantidad de entradas binarias activas en un momento dado para determinar si su salida, también binaria, debe activarse o no. Con un modelo tan simple es posible construir una red neuronal artificial¹ capaz de procesar cualquier problema lógico [24].

2.4. El *Perceptron*

El *perceptron* es un modelo de red neuronal que utiliza neuronas conocidas como TLU (*Threshold Logic Unit*). Las neuronas TLU funcionan de forma muy similar a las neuronas artificiales binarias salvo que procesan entradas no binarias x_n para dar una salida no binaria $y_{W,b}(X) = f(z)$. Dentro de la neurona se calcula una función lineal $z = \sum_1^N x_n * w_n + b$ donde cada entrada x_n es asignada un peso w_n además de utilizar un término de *offset* b . Existen diferentes funciones que transforman la combinación lineal de entradas z en la salida $y_{W,b}$ como por ejemplo: la función binaria *Heaviside*(x), la función sigmoide $\sigma(z)$, la función tangente hiperbólica $\tanh(z)$ o la función lineal rectificada $ReLU(z)$ [11, 25, 26].

Por sí mismas, ninguna neurona es capaz de aprender. Necesitan un método de ajuste para poder adaptarse al patrón subyacente de los datos. El aprendizaje de las neuronas viene dado por el error cometido en su salida $y_{W,b}(X) = \hat{y}$ con respecto a la salida esperada y . De esta forma, una red neuronal con J neuronas actualiza sus pesos $w_{i,j}$ mediante un proceso llamado entrenamiento de la forma:

$$w_{i,j}^1 = w_{i,j}^0 + \eta * (y_j^0 - \hat{y}_j^0) * x_{i,j}^0$$

Donde $x_{i,j}^0$ es la muestra actual de la entrada i de la neurona j , $(y_j^0 - \hat{y}_j^0)$ es el error cometido por la neurona j en su salida respecto a la muestra actual x^0 (nótese que las neuronas sólo tienen una salida pero I entradas), η es la tasa de aprendizaje o *learning rate*, $w_{i,j}^0$ es el peso no actualizado asignado a la entrada i de la neurona

¹En todo este trabajo se hace referencia a las redes neuronales artificiales como redes neuronales y a las neuronas artificiales como neuronas.

j en el momento de procesar la muestra $x_{i,j}^0$ y $w_{i,j}^1$ es el peso actualizado que se utilizará en la siguiente muestra de la entrada i de la neurona j [11].

2.5. Entrenamiento de redes neuronales

Un *perceptron* formado por una sola neurona puede realizar tareas simples de clasificación binaria al encontrar, tras varias muestras de entrenamiento, los pesos óptimos que minimicen el error cometido en la clasificación. De esta forma, la neurona imitaría el patrón subyacente en los datos. Un *perceptron* formado por J neuronas puede realizar una clasificación binaria en $J + 1$ clases al tener J salidas. Lo más interesante es la posibilidad de conectar las salidas de un *perceptron* a las entradas de otro *perceptron* para crear un *perceptron* multicapa.

La primera capa es llamada capa de entrada o *input layer* mientras la última capa es la capa de salida o *output layer*. El resto de capas intermedias son las *hidden layers*. Las primeras capas son capaces de reconocer las formas más básicas del patrón subyacente a los datos. Las últimas capas utilizan los patrones básicos reconocidos por las primeras para construir patrones más complejos.

Un *perceptron* multicapa forma parte de las redes neuronales profundas o *Deep Neural Networks*. El método de ajuste de pesos de una red neuronal profunda, llamado retropropagación o *backpropagation*, difiere del explicado anteriormente que involucra el uso del algoritmo conocido como descenso de gradiente o *gradient descent*.

El algoritmo *gradient descent* consiste en minimizar una función de coste cambiando iterativamente el valor de los parámetros que influyen en la salida evaluada por dicha función de coste. Cuando la red neuronal ha dado al menos dos predicciones, hay al menos dos errores medidos por la función de coste. De esta forma, se puede calcular la pendiente de la recta que une ambos errores, llamada gradiente. El algoritmo *gradient descent* utiliza el valor y signo de dicha pendiente para actualizar los pesos de las redes neuronales en el sentido descendente. De esta forma, a medida que el error se va reduciendo, se reduce también la fuerza de la actualización de pesos hasta llegar al mínimo de la función de coste donde el gradiente es nulo.

Utilizando W_t para denominar al conjunto de pesos o parámetros configurables de la red neuronal, las actualizaciones se realizan de la forma:

$$W_t = W_{t-1} - \beta * \nabla E_t(W_{t-1}) \quad (2)$$

siendo β la tasa de aprendizaje, $E_t(W_{t-1})$ el error medido por la función de coste en la muestra $t - 1$ y ∇ el operador gradiente.

Backpropagation funciona en dos fases: el pase hacia delante o *forward pass* y el pase hacia atrás *backward pass*. Durante el *forward pass*, la red neuronal procesa una entrada y guarda el valor de su salida $\hat{y}$ final junto al valor de salida de todas las neuronas que forman parte de la red. En el *backward pass* se mide el error cometido en la salida final respecto a la salida esperada y se calcula, utilizando las

salidas de las neuronas de las capas intermedias, la contribución de cada peso $w_{i,j}$ de las neuronas de las capas anteriores hasta llegar a la primera capa. Finalmente, los pesos son modificados siguiendo los principios del algoritmo *gradient descent*.

Es muy importante que los pesos $w_{i,j}$ de cada capa l sean inicializados de forma aleatoria. Si todos los pesos fueran iguales, serían actualizados en la misma cuantía por el algoritmo *gradient descent*. De esta forma la red neuronal actuaría como si cada capa sólo tuviera una neurona [11, 26, 27].

2.6. Redes neuronales convolucionales

Las redes neuronales convolucionales o CNNs (*Convolutional Neural Networks*), al igual que los *perceptrons*, imitan la estructura biológica de las neuronas. Sin embargo, las CNNs imitan la arquitectura del córtex visual. Una neurona j de la capa l no está conectada a todas las neuronas de la capa $l - 1$ sino que se conecta a un set pequeño de neuronas circundantes de la capa $l - 1$. De esta forma, juntando y solapando los campos visuales de diferentes neuronas, se llega a cubrir el campo visual entero, como si se realizara la convolución de un filtro a lo largo de una imagen. Así, las redes neuronales convolucionales son ideales para el procesamiento de datos en dos dimensiones como las imágenes.

Las capas de una red CNN disponen sus neuronas en un plano bidimensional ixj para imitar la estructura biológica. El campo visual de una neurona está definido por su altura f_h y anchura f_w . De esta forma, la neurona i, j, l , siendo está conectada a las neuronas comprendidas en el campo visual $i \pm \text{floor}(f_h/2), j \pm \text{floor}(f_w/2), l - 1$. Esto supone que las capas superiores son cada vez más pequeñas puesto que el campo visual de las neuronas cercanas a los bordes de la capa l se saldría de las dimensiones de la capa $l - 1$. Para evitarlo, es posible rodear todos los datos con ceros (*zero padding*) para que el campo visual de las capas superiores no se salga de las capas inferiores.

Aun así, para reducir la complejidad de la red neuronal, es posible conectar una capa convolucional con otra más pequeñas sin tener que estar relacionadas por los parámetros f_h y f_w al usar un salto mayor o *stride*, tanto en vertical s_h como en horizontal s_w [11, 28, 29].

2.6.1. Mapas de filtros

Durante el estudio del córtex visual biológico se descubrió que algunas neuronas sólo reconocían patrones verticales mientras otras reconocían sólo patrones en otras direcciones. Esto condujo a la creación de filtros o *kernels* en las neuronas artificiales convolucionales. Un filtro es una combinación de pesos determinada que permite a la neurona reconocer patrones en una dirección concreta. Una capa de neuronas que comparten un mismo *kernel* pertenecen a un mismo mapa de filtros. Cada neurona del mapa k de la capa l procesa todos los mapas de filtros de la capa $l - 1$ pertenecientes a su campo visual [11].

2.6.2. Capas de agregación

Las redes neuronales convolucionales presentan un gran problema en cuanto a los recursos consumidos, especialmente durante el entrenamiento. Para orquestar el algoritmo de retropropagación se necesita obtener las salidas intermedias de todas las capas de neuronas, contando con todos sus mapas de filtros. La carga computacional no es tan grande como si se tratara de una red de *perceptrons* completamente conectados. Aun así, la carga es muy grande.

Para ello se desarrollaron las capas de agregación. Su cometido es el submuestreo de los datos para reducir la carga computacional perdiendo la menor cantidad de información posible. Son capas cuyas neuronas, también dispuestas en un plano bidimensional y conectadas sólo a las neuronas dentro de su campo visual, sin pesos para sus entradas. Las neuronas de las capas de agregación combinan las entradas con funciones como máximo o promedio. Así, se reduce el tamaño de los datos a procesar eliminando la información menos importante, lo que ayuda también a que las CNNs mejoren su generalización [11].

2.7. Redes neuronales recurrentes

Las redes neuronales recurrentes o RNNs (*Recurrent Neural Networks*), a diferencia de las anteriores, no están basadas en ninguna estructura biológica. Son capas de neuronas capaces de retener información de muestras procesadas con anterioridad. Las neuronas recurrentes más simples, además de procesar las entradas x_t , procesan su propia salida de la muestra anterior $\hat{Y}_{t-1}$, con su propio peso $w_{\hat{y}}$ para generar la salida actual $\hat{Y}_t$. Esto hace a las RNN ideales para el procesamiento de secuencias de datos como, por ejemplo, series temporales.

Otras neuronas recurrentes fueron desarrolladas para aumentar la capacidad de memoria de las neuronas más simples. La mayor parte de ellas comienza por memorizar una combinación de las entradas con la salida anterior en vez de sólo la salida anterior. Esta combinación se denomina estado o *hidden state* $h_t = f(x_t, \hat{y}_{t-1})$. La salida de estas neuronas es también una función de las entradas y el estado anterior $\hat{y}_t = g(x_t, h_{t-1})$ [11, 30–32].

2.7.1. Neuronas LSTM

Aun así, con el paso de muchas entradas, la información almacenada en el estado h es desechada en favor de nueva información. Las neuronas LSTM (*Long-Short Term Memory*) fueron diseñadas para escoger si la información obtenida de las muestras procesadas más recientes es merecedora de ser almacenada en el estado de la neurona. En realidad no son en sí una neurona sino una célula compuesta de una capa de neuronas completamente conectadas y operadores de adición y multiplicación².

²Durante todo este trabajo se hará referencia a las células LSTM como neuronas LSTM.

Las neuronas LSTM dividen el estado en dos vectores: el vector de largo alcance c_t y el vector de corto alcance h_t , siendo éste último equivalente a la salida de la neurona $\hat{y}_t$. Dentro de la neurona LSTM, el vector de largo alcance sufre una pérdida de información antigua, controlada por la puerta de olvido o *forget gate* f_t , y la adición de información nueva controlada por la puerta de entrada o *input gate* i_t y la combinación g_t de la entrada x_t y el vector de corto alcance h_{t-1} .

La puerta de olvido y la puerta de entrada son controladas mediante el procesamiento del vector de corto alcance anterior h_{t-1} y las muestras de entrada actuales x_t por una capa de neuronas completamente conectadas por cada puerta. Dichas capas completamente conectadas tienen la función de activación logística, que arroja valores entre 0 y 1 en su salida. Así, controlan el porcentaje de olvido aplicado al vector c_{t-1} y el porcentaje de la combinación $g_t = g(x_t, h_{t-1})$ que se añadirá al vector c_{t-1} para formar el nuevo vector de largo alcance c_t . La combinación g_t es la salida ofrecida por una neurona recurrente básica que combina la entrada actual x_t con su estado anterior h_{t-1} .

La última parte de las neuronas LSTM es la puerta de salida o *output gate* o_t . Ésta es controlada de la misma forma que las puertas descritas anteriormente e indica qué porción del nuevo vector de largo alcance c_t es considerado como salida $\hat{y}_t$ y nuevo vector de corto alcance h_t .

En resumen, las puertas y las salidas de una neurona LSTM son controladas de la forma:

$$i_t = \sigma(W_{x,i}^T * x_t + W_{h,i}^T * h_{t-1} + b_i) \quad (3)$$

$$f_t = \sigma(W_{x,f}^T * x_t + W_{h,f}^T * h_{t-1} + b_f) \quad (4)$$

$$o_t = \sigma(W_{x,o}^T * x_t + W_{h,o}^T * h_{t-1} + b_o) \quad (5)$$

$$g_t = \tanh(W_{x,g}^T * x_t + W_{h,g}^T * h_{t-1} + b_g) \quad (6)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (7)$$

$$y_t = h_t = o_t \odot \tanh(c_t) \quad (8)$$

siendo i_t , f_t , o_t , g_t , c_t , y_t y h_t en valor en la muestra t de la puerta de entrada, olvido, salida, la combinación de la entrada x_t y h_{t-1} , el vector de largo alcance, la salida y el vector de corto alcance de la neurona LSTM; $W_{\{x,h\},\{i,f,o,g\}}$ los pesos asignados a la entrada x o al vector de corto alcance h_{t-1} de las capas completamente conectadas de las puertas; $b_{\{i,f,o,g\}}$ los valores de *offset* de las capas completamente conectadas y $\odot$ el producto elemento a elemento de matrices.

La función σ es la función sigmoide:

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

y la función $\tanh$ la función tangente hiperbólica:

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

De esta forma, las neuronas LSTM pueden reconocer una muestra de entrada relevante para construir el patrón subyacente de los datos y almacenarla a la misma vez que ignora muestras de entrada irrelevantes. Esto además permite a las neuronas LSTM encontrar patrones de largo plazo en los datos [8, 11, 30–33].

2.8. ConvLSTM

El presente trabajo analizará varios modelos con el objetivo de encontrar el patrón subyacente en varias series temporales. Dichas series representan el tráfico de red cursado en múltiples nodos situados geográficamente próximos entre sí. Aprovechando las capacidades de las redes convolucionales y recurrentes, se analizarán modelos que combinan redes CNN con RNN para encontrar el patrón espacio temporal presente en los datos esperando que ello mejore las capacidades de predicción de tráfico en red.

2.9. *Online learning*

Uno de los mayores problemas de los modelos de aprendizaje automático o *deep learning* es la evolución de los datos. Los modelos en producción suelen ser estáticos, es decir, no actualizan sus parámetros ni pesos. Esto les aporta una gran solidez pero genera una decadencia en su rendimiento a medida que los datos que procesan modifican su patrón subyacente con el paso del tiempo, alejándose del patrón aprendido por el modelo. Este problema, comúnmente denominado como *concept drift*, no siempre tiene la misma apariencia, dependiendo de los datos procesados el cambio de patrón puede ser más o menos brusco o incluso ser cíclico.

Para solucionar este problema se debe entrenar de nuevo el modelo. Esto conlleva una gran tarea por detrás. Además de tener que generar nuevos sets de entrenamiento, validación y test que representen no sólo el nuevo patrón cambiado sino también el antiguo, la inversión en tiempo y recursos de computación puede ser muy alta. Esto supone una gran inversión financiera que puede no llegar a ser rentable si la frecuencia de cambio del patrón de los datos es demasiado alta.

Como solución alternativa se propuso el entrenamiento incremental o *online learning*. El modelo procesa de forma secuencial los datos en pequeños grupos o *mini-batches* de los que aprende de forma rápida y barata. De esta forma, a medida que procesa nuevos paquetes de datos se ajusta al patrón de los nuevos datos. No todo es ventajas puesto que el modelo debe ser capaz de distinguir las irregularidades en el patrón debidas al ruido presente en los datos de las irregularidades causadas por un cambio real en el patrón [7, 34].

Los modelos basados en *online learning* son especialmente útiles para la predicción

de series temporales como el tráfico en red. Debido a la naturaleza de estas series, no necesitan la intervención ni humana ni de otros modelos para generar datos de entrenamiento. Cuando el modelo de *online learning* procesa una secuencia de datos x_t y predice la siguiente muestra $\hat{Y}_{t+1}$, no conoce el error que ha cometido en la predicción puesto que el valor real y_{t+1} es todavía desconocido. Sin embargo, llegado el momento $t + 1$ sólo es necesario medir la variable real para conocer y_{t+1} . Así se puede medir el error cometido $\epsilon_{t+1} = y_{t+1} - \hat{Y}_{t+1}$ de forma sencilla y, a pesar de la necesidad de un elemento externo para medir la variable real, autónoma.

3. Entorno experimental

3.1. Descripción de los datos

Los datos utilizados en este trabajo provienen del dataset Milán [35]. Este dataset fue creado con la intención original de descubrir la relación, si existiese, entre los eventos meteorológicos y el tráfico de red en la ciudad de Milán, Italia. El dataset contiene datos tanto climatológicos como de SMS, llamadas telefónicas y tráfico perteneciente a navegación web. Los datos están organizados en mediciones realizadas cada hora que corresponden a una celda cuadrada de $235m$ de lado dentro de un área de $553,66km^2$ de la ciudad de Milán [36]. El dataset contiene mediciones correspondientes a los meses de noviembre y diciembre de 2013. Para este trabajo se han aislado y agregado los datos relativos a SMS, llamadas telefónicas y tráfico web en muestras equiespaciadas una hora en el tiempo, desechando la información relativa a los eventos meteorológicos.

Las celdas utilizadas en este trabajo corresponden a una cuadrícula de 7×7 celdas centrada en la celda 5060. Para averiguar el alcance de las redes neuronales convolucionales, también se han realizado pruebas con una cuadrícula de 15×15 celdas centradas en la celda 5464. Esta última cuadrícula engloba completamente a la cuadrícula 7×7 .

Aproximadamente, todas las series temporales de las celdas son bastante similares entre sí. Todas presentan un patrón subyacente claro y cíclico con un periodo de más o menos 24 horas. Además, todas ellas también presentan un patrón cíclico con un periodo de una semana, en el que los primeros 5 ciclos de 24 horas tienen bastante más amplitud que los últimos 2. Esto coincide con los 5 días laborables de la semana y los 2 días del fin de semana.

Respecto a las similitudes entre celdas, es difícil afirmar que una celda tenga una serie similar a otra celda contigua a lo largo de toda la cuadrícula. Por ejemplo, la celda 5357 es contigua a las celdas 5257, 5258 y 5358 pero su serie temporal, aunque presenta las dos componentes cíclicas comunes a toda la cuadrícula, no es similar. Por otro lado, la celda 4962 es bastante similar a todas sus celdas contiguas (4861, 4862, 4863, 4961, 4963, 5061, 5062, 5063) presentando todas ellas las dos componentes cíclicas comunes de forma muy marcada.

3.2. Arquitecturas empleadas

Para este trabajo se han utilizado 5 arquitecturas de redes neuronales. En primer lugar se ha utilizado una arquitectura formada por una única capa de 24 neuronas LSTM en medio de una capa de entrada y una capa completamente conectada de 3 salidas. Con esta arquitectura se ha obtenido el rendimiento base de la propuesta realizada por el Dr. Juan Pablo Casaseca de la Higuera [8], a mejorar en este trabajo. Los datos procesados por esta arquitectura pertenecen a las celdas 4259, 4456 y 5060 del dataset Milán.

Para alcanzar el objetivo de evaluar el rendimiento en dos conjuntos de 49 y 225 celdas contiguas se han utilizado también dos arquitecturas formadas por una capa de entrada, una capa de neuronas LSTM y una capa completamente conectada de 49 y 225 salidas respectivamente. De esta forma se ha obtenido el rendimiento de la arquitectura exclusivamente LSTM con las celdas contiguas.

Para intentar extraer la correlación espacial entre las celdas, se han propuesto otras 6 arquitecturas compuestas por una capa de entrada, 1, 2 y 3 capas de neuronas CNN respectivamente, una capa **Flatten**, una capa de neuronas LSTM y una capa completamente conectada de 49 y 225 salidas respectivamente.

Para aprovechar al máximo las librerías cuDNN se ha utilizado la función de activación **sigmoide** para las puertas de entrada, salida y olvido y la función **tanh** para la combinación de entradas y estado de las neuronas LSTM [37]. Otras funciones de activación no son compatibles con los *kernel* optimizados cuDNN por lo que el tiempo de ejecución aumenta exponencialmente. Aunque cabe la posibilidad de que otras funciones de activación consigan mejores resultados, un tiempo de ejecución excesivamente alto inutiliza la red neuronal.

3.3. Procedimiento de entrenamiento

Para la evaluación y comparación de los modelos, se ha ideado un procedimiento similar a la validación de modelos de aprendizaje automático descrita en el apartado CAPÍTULO 2. Se ha entrenado los modelos con las 400 primeras muestras del dataset y se han realizado dos validaciones distintas en cada modelo. Una de las validaciones se ha realizado con las otras 1000 muestras del dataset sin utilizar entrenamiento incremental. La otra validación se ha realizado con las mismas 1000 muestras pero implementando entrenamiento incremental con ajuste de gradientes por norma euclídea del error normalizado [8] definida por la ecuación:

$$\left\| \frac{y_t - \hat{y}_t}{y_t} \right\|^2 \quad (9)$$

Y el error cuadrático como función de coste:

$$E_t(W_{t-1}) = (y_t - \hat{y}_t)^2 \quad (10)$$

De esta forma, las ecuaciones 2 y 9 se combinan en:

$$W_t = W_{t-1} - \beta * \left\| \frac{y_{t-1} - \hat{y}_{t-1}}{y_{t-1}} \right\|^2 * \nabla(y_{t-1} - \hat{y}_{t-1})^2 \quad (11)$$

De esta forma se da una mayor importancia a las predicciones que han generado

errores mayores. Estos errores dan más información y permiten realizar una actualización de pesos más agresiva para aumentar la velocidad de convergencia hacia el mínimo de la función de coste.

3.4. Hiperparámetros considerados

Hay una selección variada de parámetros configurables a la hora de entrenar las redes neuronales propuestas. No todos los parámetros han sido considerados como configurables como, por ejemplo, el tamaño de los sets de entrenamiento, test estático y test incremental. El criterio de selección se basa en el impacto de los hiperparámetros en el rendimiento de las redes neuronales en el apartado de *online learning*. Por esta razón se ha seleccionado los siguientes parámetros:

- **Tamaño de ventana deslizante:** también denominado `look_back`, además de ser el número de neuronas presentes en la capa LSTM, establece cuántas muestras de tráfico de red de cada celda son procesadas al mismo tiempo. Dichas muestras son tomadas como una misma secuencia de longitud `look_back`. Tiene un gran impacto en todas las fases de entrenamiento y test.
- **Tamaño de *batch*:** también denominado `batch_size`, indica cuántas secuencias de longitud `look_back` son procesadas por la red neuronal en un mismo momento. Tiene un gran impacto en todas las fases de entrenamiento y test.
- **n_epoch_updates:** indica la cantidad de veces que la red neuronal procesa un mismo *batch*. Cada vez que lo procesa, ajusta sus pesos si el error supera el umbral de entrenamiento incremental.
- **Umbral para el entrenamiento incremental:** también denominado `threshold`, indica el error mínimo que la red neuronal ha de cometer durante el entrenamiento incremental para ajustar sus pesos. Un mayor `threshold` elimina el ruido en los datos para evitar ajustes de pesos innecesarios aunque puede provocar que la red neuronal ignore cambios pequeños en el patrón subyacente de los datos.
- **Número de filtros de las capas CNN:** también llamado `num_filter1`, `num_filter2` o `num_filter3`, indica cuántas formas de patrones son utilizadas por las capas convolucionales para intentar componer el patrón subyacente de los datos. Tiene un gran impacto en todas las fases de entrenamiento y test.
- **Tamaño del *kernel*:** indica el tamaño del campo de visión de las neuronas convolucionales. Está estrechamente relacionado con la cantidad de celdas utilizadas en el dataset. Tiene un gran impacto en todas las fases de entrenamiento y test.

Para la búsqueda de los parámetros óptimos se ha preferido la estrategia *grid search* frente a *random search*. Los hiperparámetros considerados tienen un gran impacto en el tiempo de procesamiento de las redes neuronales, que se intenta reducir en la medida de lo posible. Esto establece un límite en el número de filtros, `n_epoch_updates`

y el tamaño de la ventana deslizante y los *batches*. Además, un umbral **threshold** demasiado alto inutiliza el entrenamiento incremental. De esta forma, aunque *random search* tenga posibilidades de probar combinaciones con valores fuera de lo normal, es casi seguro que éstos desemboquen en combinaciones que impongan una carga computacional excesiva.

Como métrica de rendimiento se ha utilizado la raíz del error cuadrático medio normalizado o NRMSE (*Root Normalized Mean Squared Error*). Sin embargo, como cada celda tiene su propio patrón temporal subyacente, cada celda tiene un NRMSE distinto. Teniendo 49 o 225 celdas es imposible comparar todos sus NRMSE con los NRMSE de otras celdas con otra combinación de hiperparámetros. Para dicha comparación, como todas las celdas son igual de importantes, se ha usado la media aritmética del NRMSE de todas las celdas.

3.5. Métricas de rendimiento

Como métrica de rendimiento se ha utilizado la raíz normalizada del error cuadrático medio o NRMSE (*Root Normalized Mean Squared Error*). Sin embargo, como cada celda tiene su propio patrón temporal subyacente, cada celda tiene un NRMSE distinto. Teniendo 49 o 225 celdas es imposible comparar todos sus NRMSE con los NRMSE de otras celdas con otra combinación de hiperparámetros. Para dicha comparación, como todas las celdas son igual de importantes, se ha usado la media aritmética del NRMSE de todas las celdas.

El NRMSE eleva al cuadrado las diferencias entre el valor real y_i y el valor predicho por la red neuronal $\hat{y}_i$ y las promedia. Finalmente calcula la raíz cuadrada del promedio y normaliza, en este trabajo, por la desviación típica de los valores reales. El NRMSE queda por tanto definido como:

$$NRMSE = \frac{\sqrt{\frac{1}{n} * \sum_{i=1}^n (y_i - \hat{y}_i)^2}}{\sigma(y)}$$

donde n es el número de muestras del conjunto medido.

El NRMSE es una métrica adimensional independiente de la magnitud de los datos que representa la distancia entre dos puntos. Es ideal para este trabajo puesto que los datos utilizados presentan grandes fluctuaciones que distorsionarían el error cuadrático medio y su raíz. Un valor de 0 de NRMSE sería perfecto, sin errores cometidos en todo el set. Sin embargo, es irracional buscar un NRMSE=0. Teniendo en cuenta los resultados obtenidos por el Dr. Juan Pablo Casaseca de la Higuera [8], el NRMSE buscado es aquel NRMSE<0.1778 en el set de entrenamiento incremental.

3.6. Análisis de Spearman

Para estimar el impacto de cada hiperparámetro en el desempeño de la red neuronal, se ha decidido optar por el análisis de Spearman. Es un método no paramétrico para la estimación de la dependencia estadística entre dos variables. A diferencia del análisis de Pearson, que busca relaciones estrictamente lineales, Spearman busca cualquier tipo de relación.

Al ser un algoritmo no paramétrico, no es necesario conocer la distribución de los datos analizados y es posible comparar variables con rangos de valores muy dispares y con muchas anomalías sin distorsionar el resultado.

3.7. Entorno de ejecución

Para el desarrollo de las redes neuronales utilizadas en este trabajo se ha utilizado una plataforma virtualizada llamada Kaggle. Ésta permite al usuario de forma gratuita el acceso a GPUs (*Graphics Processing Units*) durante 30h por semana. Las GPUs en cuestión son del modelo NVIDIA TESLA P100, ideales para el entrenamiento de redes neuronales gracias a sus 4 cores y 29 GB de RAM [38]. Además, Kaggle ofrece un entorno de edición gráfico bastante intuitivo.

El lenguaje de programación utilizado en este trabajo es Python. Es un lenguaje orientado a objetos, interpretado e interactivo. Su gran popularidad está basada en su versatilidad. La alta compatibilidad de sus módulos ha hecho que existan numerosas librerías y paquetes especializados y eficientes en multitud de tareas entre las cuales se encuentra el desarrollo de redes neuronales como Tensorflow o SKLearn.

En *frameworks* de computación en paralelo como CUDA (*Compute Unified Device Architecture*), los cálculos realizados por las redes neuronales se reparten entre diferentes hilos. Cada hilo procesa los datos en momentos de tiempo diferentes según el estado del organizador de eventos del sistema operativo. Por lo tanto, en cada entrenamiento se consiguen resultados distintos puesto que el estado del sistema operativo cambia y las pequeñas diferencias en el tiempo de ejecución de los hilos se acumulan a lo largo de todo el entrenamiento o test. Además, el lenguaje de programación en el que se basa este trabajo, Python, y las librerías Tensorflow y cuDNN inician sus algoritmos de forma aleatoria.

Para conseguir la reproducibilidad de los resultados se ha de introducir varias restricciones para eliminar la aleatoriedad. Se han fijado todas las semillas de generación de números pseudoaleatorios de Python, Tensorflow y NumPy y se ha forzado a las librerías Tensorflow y cuDNN a acceder a los recursos hardware de forma determinista a través de las variables de entorno del sistema operativo [39, 40].

4. Resultados y discusión

4.1. Desempeño general

4.2. Análisis de Spearman

4.3. Mejores modelos

5. Conclusiones y líneas futuras

Referencias

- [1] G. Bell, “Bell’s law for the birth and death of computer classes,” *Communications of the ACM*, vol. 51, no. 1, pp. 86–94, Jan 2008. [Online]. Available: <https://gordonbell.azurewebsites.net/cacm%20bell’s%20law%20vol%2051.%202008-january.pdf>
- [2] A. Azzouni and G. Pujolle, “A long short-term memory recurrent neural network framework for network traffic matrix prediction,” *arXiv preprint arXiv:1705.05690*, Jun 2017. [Online]. Available: <http://arxiv.org/abs/1705.05690>
- [3] Y. S. Abu-Mostafa, M. Magdon-Ismail, and H. T. Lin, *Learning from Data: A Short Course*. Amlbook.com, 2012.
- [4] N. Bui, M. Cesana, S. A. Hosseini, Q. Liao, I. Malanchini, and J. Widmer, “A survey of anticipatory mobile networking: context-based classification, prediction methodologies, and optimization techniques,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 3, pp. 1790–1821, 2017.
- [5] T. Guo and X. Xu, “Robust online time series prediction with recurrent neural networks,” in *2016 IEEE International Conference on Data Mining (ICDM)*, 2016, pp. 1001–1006.
- [6] Y. Duan, G. Fu, and N. Zhou, “Everything as a service(xaas) on the cloud: Origins, current and future trends,” in *2015 IEEE 8th International Conference on Cloud Computing*. IEEE Computer Society, 2015, pp. 621–628.
- [7] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, “Learning under concept drift: A review,” *IEEE Transactions on Knowledge and Data Engineering*, 2018, arXiv:2004.05785.
- [8] P. Casaseca-de-la Higuera, J. M. Aguiar, P. Amado-Caballero, A. J. Morgado, J. d. S. Moreira, C. Luo, and X. Wang, “Online active learning for lstm-based network traffic prediction,” in *IEEE (Unpublished Draft / Preprint)*, 20XX.
- [9] M. González and I. M. del Puerto, *Series Temporales*, ser. Colección manuales uex. Universidad de Extremadura, 2009, vol. 60.
- [10] J. C. García Díaz, *Predicción en el dominio del tiempo: análisis de series temporales para ingenieros*, ser. Manual de referencia. Valencia: Editorial de la Universidad Politécnica de Valencia, 2016.
- [11] A. Géron, *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems*, 3rd ed. Beijing: O’Reilly, 2022.
- [12] P. K. Hoong, I. K. T. Tan, and C. Y. Keong, “Bittorrent network traffic forecasting with arma,” *International journal of Computer Networks & Communications*, vol. 4, pp. 143–156, Jul 2012.

- [13] H. O. A. Wold, *A Study in the Analysis of Stationary Time Series*. Uppsala, Sweden: Almqvist & Wiksell, 1938. [Online]. Available: <https://archive.org/details/in.ernet.dli.2015.262214>
- [14] K. Poo, I. K. T. Tan, and K. Ng, “Computing autoregressive moving average (arma) with cronos: A case study of bursa malaysia,” in *2nd International Conference on Engineering & ICT 2010 (ICEI)*, Melaka, Malaysia, 2010, pp. 516–520.
- [15] M. Papadopouli, H. Shen, E. Raftopoulos, M. Ploumidis, and F. H. Campos, “Short term traffic forecasting in a campus wide wireless network,” in *16th IEEE Annual International Symposium on Personal Indoor and Mobile Radio Communications*, Berlin, Germany, Sep 2005.
- [16] S. Jung, C. Kim, and Y. Chung, “A prediction method of network traffic using time series models,” in *Computational Science and Its Applications (ICCSA)*, ser. Lecture Notes in Computer Science (LNCS). Springer, 2006, pp. 234–243.
- [17] Y. Yu, J. Wang, M. Song, and J. Song, “Network traffic prediction and result analysis based on seasonal arima and correlation coefficient,” in *2010 International Conference on Intelligent System Design and Engineering Application*, vol. 1, 2010, pp. 980–983.
- [18] T. M. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997. [Online]. Available: <http://www.cs.cmu.edu/~tom/mlbook.html>
- [19] A. L. Samuel, “Some studies in machine learning using the game of checkers,” *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, Jul 1959. [Online]. Available: <https://ieeexplore.ieee.org/document/5392560>
- [20] Y. S. Abu-Mostafa, M. Magdon-Ismael, and H. T. Lin, *Learning from Data: A Short Course*. Amlbook.com, 2012.
- [21] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012. [Online]. Available: <http://jmlr.org/papers/v13/bergstra12a.html>
- [22] D. H. Wolpert, “The lack of a priori distinctions between learning algorithms,” *Neural Computation*, vol. 8, no. 7, pp. 1341–1390, 10 1996. [Online]. Available: <https://doi.org/10.1162/neco.1996.8.7.1341>
- [23] L. N. Smith, “A disciplined approach to neural network hyper-parameters: Part 1 - learning rate, batch size, momentum, and weight decay,” *CoRR*, vol. abs/1803.09820, 2018. [Online]. Available: <http://arxiv.org/abs/1803.09820>
- [24] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, Dec 1943. [Online]. Available: <https://doi.org/10.1007/BF02478259>
- [25] F. Rosenblatt, “The perceptron: A perceiving and recognizing automaton (project PARA),” Cornell Aeronautical Laboratory, Buffalo, NY, Report 85-460-1, Jan 1957. [Online]. Available: <https://bpb-us-e2.wpmucdn.com/websites.umass.edu/dist/a/27637/files/2016/03/rosenblatt-1957.pdf>

- [26] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning internal representations by error propagation,” Institute for Cognitive Science, University of California, San Diego, La Jolla, CA, Technical Report ICS-8506, Sep 1985. [Online]. Available: <https://apps.dtic.mil/sti/citations/ADA164453>
- [27] —, “Learning internal representations by error propagation,” Institute for Cognitive Science, University of California, San Diego, CA, Technical Report ICS-8506, Sep 1985. [Online]. Available: <https://apps.dtic.mil/sti/citations/ADA164453>
- [28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov 1998. [Online]. Available: <https://ieeexplore.ieee.org/document/726791>
- [29] K. Fukushima, “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position,” *Biological Cybernetics*, vol. 36, no. 4, pp. 193–202, Apr 1980. [Online]. Available: <https://doi.org/10.1007/BF00344251>
- [30] V. Pham, T. Bluche, C. Kermorvant, and J. Louradour, “Dropout improves recurrent neural networks for handwriting recognition,” in *14th International Conference on Frontiers in Handwriting Recognition (ICFHR)*. IEEE, Sep 2014, pp. 285–290. [Online]. Available: <https://ieeexplore.ieee.org/document/6981034/>
- [31] Q. V. Le, N. Jaitly, and G. E. Hinton, “A simple way to initialize recurrent networks of rectified linear units,” 2015. [Online]. Available: <https://arxiv.org/abs/1504.00941>
- [32] N. Ramakrishnan and T. Soni, “Network traffic prediction using recurrent neural networks,” in *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*. Orlando, FL, USA: IEEE, Dec 2018, pp. 187–193. [Online]. Available: <https://ieeexplore.ieee.org/document/8614060/>
- [33] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [34] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, Mar 2014. [Online]. Available: <https://dl.acm.org/doi/epdf/10.1145/2523813>
- [35] T. Italia, “Telecommunications - SMS, Call, Internet - MI,” 2015. [Online]. Available: <https://doi.org/10.7910/DVN/EGZHFV>
- [36] —, “Milano Grid,” 2015. [Online]. Available: <https://doi.org/10.7910/DVN/QJWLFU>
- [37] Keras Team. (2024) LSTM layer: Keras recurrent layers API. [Accessed: Jun. 20, 2026]. [Online]. Available: https://keras.io/api/layers/recurrent_layers/lstm/

- [38] Kaggle. (2026) Kaggle notebooks documentation. [Accessed: Jun. 19, 2026]. [Online]. Available: <https://www.kaggle.com/docs/notebooks>
- [39] TensorFlow Developers. (2022, May) What's new in TensorFlow 2.9. [Accessed: Jun. 19, 2026]. [Online]. Available: <https://blog.tensorflow.org/2022/05/whats-new-in-tensorflow-29.html>
- [40] NVIDIA Corporation. (2026) cudnn backend API developer guide: Miscellaneous. [Accessed: Jun. 19, 2026]. [Online]. Available: <https://docs.nvidia.com/deeplearning/cudnn/backend/latest/developer/misc.html>